

ALERT-03: Routing, Destinations, and Cost

Series: ALERT — Alerting Strategy and Design | **Notebook:** 03 of 05 | **Created:** June 2026 | **Last Updated:** 07/21/2026

Overview

Once a problem fires, getting it to the right people without waste is a routing problem with a cost dimension. This notebook covers the **simple vs multi-step workflow** decision (and its billing implications), the destination landscape, and where the legacy alerting-profile path still applies. It orchestrates the WFLOW series rather than repeating it.

Table of Contents

- [1. Simple vs Multi-Step Workflows](#)
 - [2. The Routing Pattern](#)
 - [3. Destination Landscape](#)
 - [4. The Legacy Path](#)
 - [5. Cost Discipline](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3 with AutomationEngine (Workflows)

Requirement	Details
Prior reading	ALERT-01; routing depth in WFLOW-03/04
Upstream	Problems enriched with routing metadata (ALERT-01 §4)

1. Simple vs Multi-Step Workflows

This is the cost decision that the field most often gets wrong.

	Simple workflow	Multi-step workflow
Shape	Problem trigger → one notification action	Trigger → conditions, multiple actions, enrichment, multiple teams
Billing	Not charged on run (query + send)	Charged (workflow-hours)
Use when	Filter problems and send to one channel	Routing to multiple teams, leaving comments, conditional logic, enrichment, centralised config

Default to simple workflows where possible. This may mean several simple workflows instead of one big one — that is the cheaper shape, because their query and runs are not charged under workflow-hours. Reach for a multi-step workflow when you genuinely need conditional branching, multi-team routing, or a single centralised configuration; the capability is worth the cost when you need it, but do not pay it by default.

Verify current billing specifics against the Workflows documentation for your DPS model — the simple-vs-billed boundary is the kind of detail that shifts, and it is exactly what the field-authored source document got partially right but did not fully pin down.

A worked trap. A frequent pattern is *one routing workflow per team or area* that catches every problem for that area and posts to the team's chat channel. It looks like simple fan-out — but the form of the delivery action decides the billing, not the apparent shape. A single built-in notification action keeps the workflow simple; a hand-rolled HTTP-function call or a run-JavaScript task to reach the same channel is what

can tip it into the billed multi-step category. Choose the action type deliberately, and confirm which connectors count as simple notification actions for your DPS model.

2. The Routing Pattern

1. Create a workflow with a **Problem trigger**.
2. Configure the trigger to **filter the problems** relevant to this channel — filter on the metadata you enriched upstream (team, zone, service, severity).
3. Add the **notification action** for the channel.
4. Set up the **connection** to that channel if not already present.
5. Compose the message, using `{` to embed problem details (name, link, severity, affected entity).

Routing dimensions — severity, team/ownership, service, time of day — and escalation patterns are covered in depth in WFLOW-04. Sprint-1.337 made Smartscape ownership a first-class routing attribute, so workflows can read the owning team directly rather than maintaining a side-table.

3. Destination Landscape

Destination	Path	Notebook
Slack / Teams	Native workflow connector	WFLOW-03/04
PagerDuty / on-call	Native connector — use for fast-burn pages	WFLOW-04, SLO-04
Jira	Native connector — create/comment/assign issues	WFLOW-04
Jira Service Management	Native JSM connector (SaaS 1.343, staged rollout — verify in tenant) — send Dynatrace events to JSM for alert management	WFLOW-04
ServiceNow	Native connector / HTTP Table API / ITOM app	ALERT-04
Email	Native action	WFLOW-03
xMatters	Legacy alerting-profile path (see below)	—

Destination	Path	Notebook
Anything else	HTTP action to a webhook	WFLOW-08

Match the destination to the urgency: fast-burn / acute → page (PagerDuty, on-call); slow-burn / steady → ticket (Jira, ServiceNow).

Migrating a classic notification: the official mapping

If you are converting classic problem notifications to workflows, this is the documented old→new correspondence:

Classic notification	Workflow equivalent
Ansible	RedHat Ansible connector
Custom integration	HTTP Request action
Email	Microsoft 365 / Email connector
Jira	Jira connector
PagerDuty	PagerDuty connector
ServiceNow	ServiceNow connector
Slack	Slack connector

The four with no native connector

Opsgenie, Trello, VictorOps, and xMatters are currently not supported as native workflow connectors. Each becomes an HTTP-action rebuild against the destination's current API — and per §1, adding an HTTP action can move a workflow out of the *simple* (unbilled) category into the billed multi-step one. Budget for that rather than discovering it.

Rebuild the payload against the destination's live API contract; do not port the classic webhook body verbatim.

⚠ A classic integration scoped by a Management Zone is only as durable as that Management Zone. The MZ filter has **no successor** inside the alerting model — Dynatrace's upgrade guide states it is *"no longer supported."* If you are retiring Management Zones, those notifications must be rebuilt as problem-

triggered workflows first, filtered on affected-entity tags. **MZ2POL-09** covers that conversion end to end, including the capability regressions.

Forthcoming/rolling out (SaaS 1.343): a dedicated **Jira Service Management connector** joins the destination landscape — it sends Dynatrace events into JSM's alert management, distinct from the existing Jira (issue-tracking) connector. SaaS 1.343 released July 7, 2026 with a **staged tenant rollout** (from mid-July 2026) — verify the feature has reached your tenant before relying on it. Once available, it becomes the first-choice JSM path; until then, the existing Jira connector and custom-webhook paths described in this section remain the working options.

4. The Legacy Path

Some integrations predate workflows and still rely on **classic problem notifications** driven by alerting profiles rather than the AutomationEngine. **xMatters** is the canonical example: Dynatrace workflows are the preferred routing method, but they do not drive xMatters directly unless you POST the problem JSON to an xMatters endpoint via a workflow HTTP action.

If you have a classic problem-notification integration working, it is not deprecated — but new integrations should use workflows. The one caveat that does bite: if the integration is scoped by a **Management Zone**, it inherits that zone's lifetime (see §3). When you find a destination that "only works the classic way," check whether it exposes a webhook a workflow HTTP action can target before settling for the alerting-profile path.

5. Cost Discipline

- **Prefer simple workflows.** Several cheap simple workflows beat one billed multi-step workflow when no conditional logic is needed.
- **One problem, one notification.** Let the Davis problem group related signals; do not also alert on the underlying raw metrics, or you double-notify and double-spend.
- **Filter early in the trigger.** A trigger that matches every problem and decides relevance later still evaluates on every problem.

- **Centralise only when it pays.** A single multi-step routing workflow is easier to govern but is billed; weigh that against many simple ones.

Sources: [Workflows \(DT docs\)](#), [Workflow actions — Jira / MS Teams \(DT docs\)](#). **Softened:** exact simple-vs-billed boundaries follow your DPS model — verify against current Workflows billing docs.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.